CWasm
(Diego's version)

Diego Oniarti

1 Values

A new value is introduced that is the colored pointer (here referred to just pointer for brevity). Pointers are kin to CHERI capabilities and have three components:

1. addr: An address in memory
2. color: Pointers are used to access the new "colored memory". The color of a pointer must match the color of the memory region it tries to access, lest it traps.
3. tag: An identifier given to the pointer upon allocation. This id is preserved when the pointer is incremented and decremented.

For now we assume infinite possible colors, represented by natural numbers. In later stages of the project I'm going to limit the number of colors.

$$
\begin{array}{rl}
 & \texttt{byte} ::= \texttt{0x00} \mid \cdots \mid \texttt{0xff} \\
\text{(integers)} & \begin{cases} \texttt{uN} ::= 0 \mid 1 \mid \cdots \mid 2^N - 1 \\ \texttt{sN} ::= -2^{N-1} \mid \cdots \mid -1 \mid 0 \mid 1 \mid \cdots \mid 2^{N-1} - 1 \\ \texttt{iN} ::= \texttt{uN} \end{cases} \\
 & \texttt{name} ::= \texttt{char}^* \\
 & \texttt{char} ::= \texttt{U} + \texttt{00} \mid \cdots \mid \texttt{U} + \texttt{D7FF} \mid \texttt{U} + \texttt{E000} \mid \cdots \mid \texttt{U} + \texttt{10FFFF} \\
\text{(pointer)} & \texttt{p} ::= \{\texttt{addr} : \texttt{u32},\ \texttt{color} : \mathbb{N},\ \texttt{id} : \mathbb{N}\} \\
\text{(value)} & \texttt{v} ::= \tau_{\texttt{n}}.\texttt{const c} \mid \tau_{\texttt{ptr}}.\texttt{const p} \\
 & \quad \ldots \text{floats and others unchanged} \ldots
\end{array}
$$

2 Types

A new type for pointers has been introduced (τ_{ptr}), while the old numeric type get relegated to the "numeric type" τ_n. The more generic τ takes both classes of types.

$$
\begin{aligned}
&\text{(numeric)} && \tau_{\mathtt{n}} ::= \mathtt{i32} \mid \mathtt{i64} \mid \mathtt{f32} \mid \mathtt{f64} \\
&\text{(value type)} && \tau ::= \tau_{\mathtt{n}} \mid \tau_{\mathtt{ptr}} \\
&\text{(function/instructions)} && \rho ::= \tau^* \to \tau^* \\
& && \quad \ldots rest \ldots
\end{aligned}
$$

3 Runtime instances

The "colored memory" is introduced as an alternative to the standard heap. Bytes in colored memory are "tagged" with a color and a tag. The color is set to an area of memory when it is allocated, and it is used to ensure spatial safety. The tag distinguishes between numeric values and handles. New instructions are used to load and store handles to memory, and a load will trap if the memory is not tag as containing a pointer.

$$
\begin{aligned}
&\text{(heap)} && \mathtt{H} ::= \{\mathtt{data} : \mathtt{byte}^*,\ \mathtt{max} : \mathtt{u32}^?\} \\
&\text{(tagged byte)} && \mathtt{tbyte} ::= \mathtt{byte} \times \mathbb{N} \times \tau_{\mathtt{tag}} \\
&\begin{pmatrix}\text{colored memory} \\ \text{instance}\end{pmatrix} && \Xi ::= \{\mathtt{data} : \mathtt{tbyte}^*,\ \mathtt{max} : \mathtt{u32}^?\} \\
&\text{(allocator instance)} && \mathtt{A} ::= \mathtt{to\ be\ determined} \\
&\text{(num/ptr)} && \tau_{\mathtt{tag}} ::= \circ \mid \square \\
&\text{(store)} && \ldots
\end{aligned}
$$

The store gets extended with an instance of the colored memory and an instance of the allocator

4 Instructions

Instructions defined with reference to both Wasm and the Iris-MSWasm paper.

4.1 Numeric instructions

Numeric instructions are modified to only work on numeric types (τ_n) instead of all types (τ).

4.2 Parametric

Parametric instructions are unchanged.

4.3 Variable instructions

Variable instructions are also unchanged. Pointers can be set and gotten like any other value.

4.4 Administrative instructions

The modifications to the administrative instructions are the same as in the iris-MSWasm paper.

$$\texttt{ai} ::= \texttt{bi} \mid \tau_{\texttt{ptr}}.\texttt{const p} \mid \texttt{trap} \mid \texttt{invoke i} \mid \ldots$$

4.5 Memory instructions

The existing memory instructions work as usual for the types already existing in Wasm. $\tau_{ptr}.load$ will trap to avoid forgery. $\tau_{ptr}.store$ could still be allowed.

$$\text{(basic instructions)} \qquad \begin{aligned} \texttt{bi} ::= \; & \cdots \mid \texttt{colalloc} \mid \texttt{colfree} \mid \tau_{\texttt{ptr}}.\texttt{add} \\ & \mid \tau.\texttt{colstore} \mid \tau.\texttt{colload} \end{aligned}$$

- $\texttt{colalloc}$: pops an int n from the stack and allocate n bytes of memory in the colored memory Ξ. The allocated bytes need to be zeroed and colored with a fresh color. The instruction puts a pointer $\texttt{p}$ on the stack. The pointer has a fresh id and the same color as the memory.

- $\texttt{colfree}$: pops a $\tau_{\texttt{ptr}}$ from the stack. The id of the pointer is invalidated by the allocator, which will make all subsequent uses of it trap. [1]

- $\tau_{\texttt{ptr}}.\texttt{add}$: pops a $p : \tau_{\texttt{ptr}}$ and a $n : \texttt{i32/i64}$ from the stack. Adds n to the address component of p. Incrementing (or decrementing) a pointer address outside of its allocated area does not cause a trap. Only utilizing the pointer (load/store) will trap.

- $\tau.\texttt{colstore}$: pops a pointer and a value from the stack. Interprets the value as τ and writes its byte representation to the colored memory.
  If τ is τ_{ptr}, the value must be a $\tau_{\texttt{ptr}}.\texttt{const p}$. If τ is τ_n, the value must be $\tau_{\texttt{n}}$.
  All bytes being accessed in memory must have the same color as the pointer. The appropriate tag will be set the altered bytes, depending on τ.

- $\tau.\texttt{colload}$: Pops a pointer from the stack, loads the value in colored memory that pointer points to, and pushes the value onto the stack. If τ is τ_{ptr} all bytes read from memory need to have the $\square$ tag.

[1] The instructions as described here do not take into consideration the allocator strategy. New allocations should be able to overwrite freed areas of memory